

groupby

continent	iso_code	location	date	total_cases	new_cases
Africa	ZWE	Zimbabwe	12/11/2020	752269	13944
Asia	YEM	Yemen	12/11/2020	8874290	97894
Europe	VAT	Vatican	12/11/2020	1991233	86852
North America	VIR	United States Virgin Islands	12/11/2020	11205486	184813
Oceania	WLF	Wallis and Futuna	12/11/2020	27750	1384
South America	VEN	Venezuela	12/11/2020	5876464	69074

groupby

new_cases_smoothed	total_deaths	new_deaths	new_deaths_smoothed	total_cases_per_million
12583.714	20314	572	297.429	17698.224
93198.571	130519	2003	1168	49884.137
54868.571	52147	2004	1101	76541.772
156419.143	247220	4928	2715.143	43721.784
551.714	907	59	22	41672.6
46393	166014	3935	1096.714	29170.225

groupby

new_cases_per_million	new_cases_smoothed_per_million	total_deaths_per_million
1247.34	212.173	342.513
1763.437	800.091	611.157
8652.658	2472.188	1248.014
1648.465	844.303	766.764
4926.95	1389.9	580.627
1892.582	539.049	1069.731

groupby

new_deaths_per_million	new_deaths_smoothed_per_million	reproduction_rate	icu_patients
22.809	7.639	2.83	0
111.585	22.75	6.72	0
176.793	63.14	4.31	7019
200.04	28.577	3.73	15130
35.599	10.157	2.4	0
215.382	33.7	3.03	0

groupby

icu_patients_per_million	hosp_patients	hosp_patients_per_million	weekly_icu_admissions
0	0	0	0
0	0	0	0
126.838	33004	773.463	4375.407
45.71	65368	197.485	1943
0	0	0	0
0	0	0	0

groupby		
weekly_icu_admissions_per_million	weekly_hosp_admissions	weekly_hosp_admissions_per_million
0	0	0
0	0	0
192.21	46641.41	2424.482
5.87	28388	85.764
0	0	0
0	0	0

groupby

total_tests	new_tests	total_tests_per_thousand	new_tests_per_thousand	new_tests_smoothed
5130192	56663	98.788	0.955	47675
160000000	1492409	1523.049	15.66	1196972
69111898	648370	1926.641	26.071	557506
166039068	1531394	501.625	4.627	1296553
9379339	94706	367.819	5.185	75948
6421441	42071	251.862	2.201	98318

groupby

new_tests_smoothed_per_thousand	tests_per_case	positive_rate	stringency_index	population
0.804	4116	0.551	100	206139587
15.857	45864	0.425	100	1.439E+09
19.172	6223	0.47	100	145934460
3.917	1449	0.534	100	331002647
4.348	27062	0.143	96.3	25499881
1.821	2712.5	0.651	100	212559409

groupby

population_density	median_age	aged_65_older	aged_70_older	gdp_per_capita	extreme_poverty
622.962	37.4	10.945	5.884	26382.287	77.6
7915.731	48.2	27.049	18.493	116935.6	30.3
19347.5	47.9	23.021	16.24	94277.965	5.7
1308.82	43.1	18.601	10.799	54225.446	23.5
304.128	37.9	15.504	10.129	44648.71	25.1
66.939	35.6	14.655	10.361	22767.037	7.1

groupby				
cardiovasc_death_rate	diabetes_prevalence	female_smokers	male_smokers	handwashing_facilities
525.432	22.02	9.7	65.8	89.827
724.417	17.72	26.9	78.1	98.999
539.849	10.08	44	58.3	97.719
430.548	17.11	19.1	53.3	90.65
561.494	30.53	23.5	48.8	82.502
373.159	12.54	34.2	42.9	80.635

groupby

hospital_beds_per_thousand	life_expectancy	human_development_index	month
6.3	76.88	0.797	12
13.05	84.86	0.933	12
13.8	86.75	0.953	12
5.8	83.92	0.926	12
3.84	83.44	0.939	12
5	81.44	0.843	12

groupby

```
total_deaths_to_total_cases
0.027003638
0.014707543
0.026188296
0.022062408
0.032684685
0.028250662
```

Part 1: COVID-19 Data Analysis

Internship Studio — Data Science Project

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

import warnings
warnings.filterwarnings('ignore')
```

import dataset

```
url = "https://raw.githubusercontent.com/SR1608/Datasets/main/covid-data.csv" df =
pd.read_csv(url)

df

print("=" * 60)

print("data loaded successfully")

("\n - 2a.shape (rows, columns) -")

df.shape

df.dtypes

df.info()

df.describe()

df['location'].nunique()

df['continent'].value_counts().idxmax()

df['continent'].value_counts()

df['total_cases'].max()

print(f" Mean : {df['total_cases'].mean():.2f}")

df['total_deaths'].quantile([0.25,0.50,0.75])

idx = df['human_development_index'].idxmax()

df.loc[idx, 'continent']
```

filter columns

```
cols = ['continent', 'location', 'date', 'total_cases', 'total_deaths', 'gdp_per_capita', 'human_d
evelopment_index']

df = df[cols]
```

```
df.shape
```

data cleaning

```
before = len(df) df.drop_duplicates(inplace=True) print(f" Removed: {before - len(df)}  
duplicate rows")
```

```
df.isnull().sum()
```

```
df.dropna(subset=['continent'], inplace=True) print(f" Shape after drop: {df.shape}")
```

```
df.shape
```

```
df.fillna(0, inplace=True) print(df.isnull().sum())
```

date time format

```
df['date'] = pd.to_datetime(df['date']) df['month'] = df['date'].dt.month  
df[['date', 'month']].head()
```

data aggregation

```
df_groupby = df.groupby('continent').max().reset_index()
```

```
df_groupby
```

feature engineering

```
df_groupby['total_deaths_to_total_cases'] = ( df_groupby['total_deaths'] /  
df_groupby['total_cases'] ) print(df_groupby[['continent', 'total_deaths_to_total_cases']])
```

data visualization

```
sns.set_theme(style = "darkgrid")
```

```
fig, ax = plt.subplots(figsize=(8, 5)) sns.histplot(df_groupby['gdp_per_capita'], kde=True,  
ax=ax, color='steelblue') ax.set_title('Distribution of GDP per Capita') ax.set_xlabel('GDP  
per Capita') plt.tight_layout() plt.savefig('C:/Users/USER/Desktop/data analyst  
project/histogram.png', dpi=150) plt.close() print("Histogram Saved")
```

```
fig, ax = plt.subplots(figsize=(8, 5)) sns.scatterplot(data=df_groupby, x='gdp_per_capita',  
y='total_cases', hue='continent', s=120, ax=ax) ax.set_title('Total Cases vs GDP per  
Capita') plt.tight_layout() plt.savefig('C:/Users/USER/Desktop/data analyst  
project/scatter.png', dpi=150) plt.close() print("scatterPlot Saved")
```

```
pairplot_cols = ['total_cases', 'total_deaths', 'gdp_per_capita', '  
human_development_index'] pair_fig = sns.pairplot(df_groupby[pairplot_cols +  
['continent']], hue='continent', corner=True) pair_fig.figure.suptitle('Pairplot of  
df_groupby', y=1.01) pair_fig.savefig('C:/Users/USER/Desktop/data analyst  
project/pairplot.png', dpi=150) plt.close() print("pairplot saved")
```

```
cat = sns.catplot(data=df_groupby, x='continent', y='total_cases', kind='bar', height=5,  
aspect=1.6, palette='tab10') cat.set_xticklabels(rotation=30, ha='right')  
cat.figure.suptitle('Total Cases by Continent', y=1.02) plt.tight_layout()  
cat.savefig('C:/Users/USER/Desktop/data analyst project/barplot.png', dpi=150) plt.close()  
print(" Bar plot saved")
```

```
out_csv = 'C:/Users/USER/Downloads/sales_dashboard_large.csv' df_groupby.to_csv(out_csv,  
index=False) print(f"\n■■ 10. df_groupby saved to {out_csv} ■■")
```

Part 2: Loan Dataset — ML Classification

Internship Studio — Data Science Project

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

import warnings
warnings.filterwarnings('ignore')

from sklearn.linear_model import LogisticRegression

from sklearn.model_selection import train_test_split

from sklearn.metrics import (accuracy_score, confusion_matrix, classification_report,
                             ConfusionMatrixDisplay)
from sklearn.preprocessing import LabelEncoder
```

load dataset

```
df = pd.read_csv('C:/Users/USER/Downloads/loan_data_set_lyst1728376986423.csv')

print("Loan DataSet loaded")

print(f"Shape: {df.shape}")

df.head()
```

data understanding

```
df.dtypes

df.describe()

df.isnull().sum()
```

data cleaning

```
df = df.drop('Loan_ID', axis=1, errors='ignore')

df
```

fill categorical with mode (using correct pandas cow syntax)

```
cat_fill = ['Gender', 'Married', 'Dependents', 'Self_Employed']
for col in cat_fill:
    df[col] = df[col].fillna(df[col].mode()[0])
```

fill numerical with median


```
for col in ['LoanAmount', 'Loan_Amount_Term', 'Credit_History']: df[col] =  
df[col].fillna(df[col].median())
```

```
print("Missing values after cleaning:") df.isnull().sum()
```

EDA Visualizations

```
sns.set_theme(style="whitegrid") fig, axes = plt.subplots(2, 3, figsize=(16, 10))  
fig.suptitle('Loan Dataset - EDA', fontsize=16, fontweight='bold') sns.countplot(data=df,  
x='Loan_Status', palette='Set2', ax=axes[0, 0]) axes[0, 0].set_title('Loan Status  
Distribution') sns.countplot(data=df, x='Gender', hue='Loan_Status', palette='Set1',  
ax=axes[0, 1]) axes[0, 1].set_title('Loan Status by Gender') sns.histplot(df['LoanAmount'],  
kde=True, color='steelblue', ax=axes[0, 2]) axes[0, 2].set_title('Loan Amount  
Distribution') sns.scatterplot(data=df, x='ApplicantIncome', y='LoanAmount',  
hue='Loan_Status', alpha=0.6, ax=axes[1, 0]) axes[1, 0].set_title('Income vs Loan Amount')  
sns.countplot(data=df, x='Credit_History', hue='Loan_Status', palette='Set2', ax=axes[1,  
1]) axes[1, 1].set_title('Credit History vs Loan Status') sns.countplot(data=df,  
x='Property_Area', hue='Loan_Status', palette='Set3', ax=axes[1, 2]) axes[1,  
2].set_title('Property Area vs Loan Status')
```

```
plt.tight_layout() plt.savefig('Loan_DataSet EDA.png', dpi=150, bbox_inches='tight')  
plt.close() print("\n■■■ EDA plots saved ■■■")
```

Encode Categorical Variable

```
le = LabelEncoder() cat_cols = ['Gender', 'Married', 'Dependents', 'Education', '  
Self_Employed', 'Property_Area', 'Loan_Status'] for col in cat_cols: df[col] =  
le.fit_transform(df[col].astype(str))
```

```
df.head()
```

```
print("Verify no NAN remains") df.isnull().sum()
```

feature selection and train-test split

```
X = df.drop('Loan_Status', axis=1) y = df['Loan_Status'] X_train, X_test, y_train, y_test =  
train_test_split( X, y, test_size=0.2, random_state=42, stratify=y ) print(f"\nTrain size:  
{X_train.shape}, Test size: {X_test.shape}")
```

Logistic Regression Model

```
model = LogisticRegression(max_iter=1000, random_state=42) model.fit(X_train, y_train)  
y_pred = model.predict(X_test)
```

```
from sklearn.preprocessing import StandardScaler scaler = StandardScaler() X_train =  
scaler.fit_transform(X_train) X_test = scaler.transform(X_test)
```

model evaluation

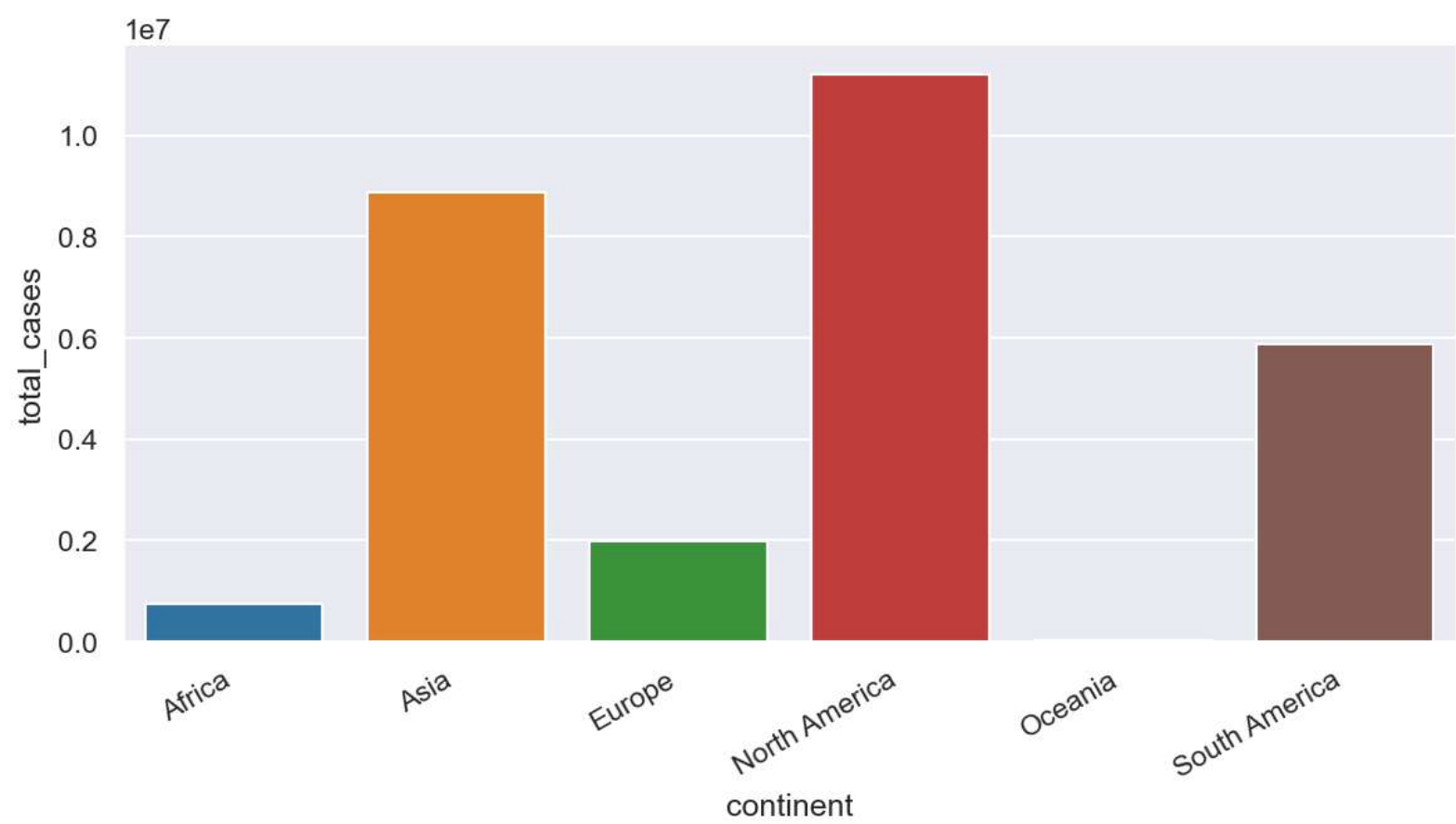
```
acc = accuracy_score(y_test, y_pred) print(f"\n Model Accuracy: {acc * 100:.2f}%")  
print("\n■■■ Classification Report ■■■") print(classification_report(y_test, y_pred,  
target_names=['Rejected (N)', 'Approved (Y)']))
```

Confusion matrix

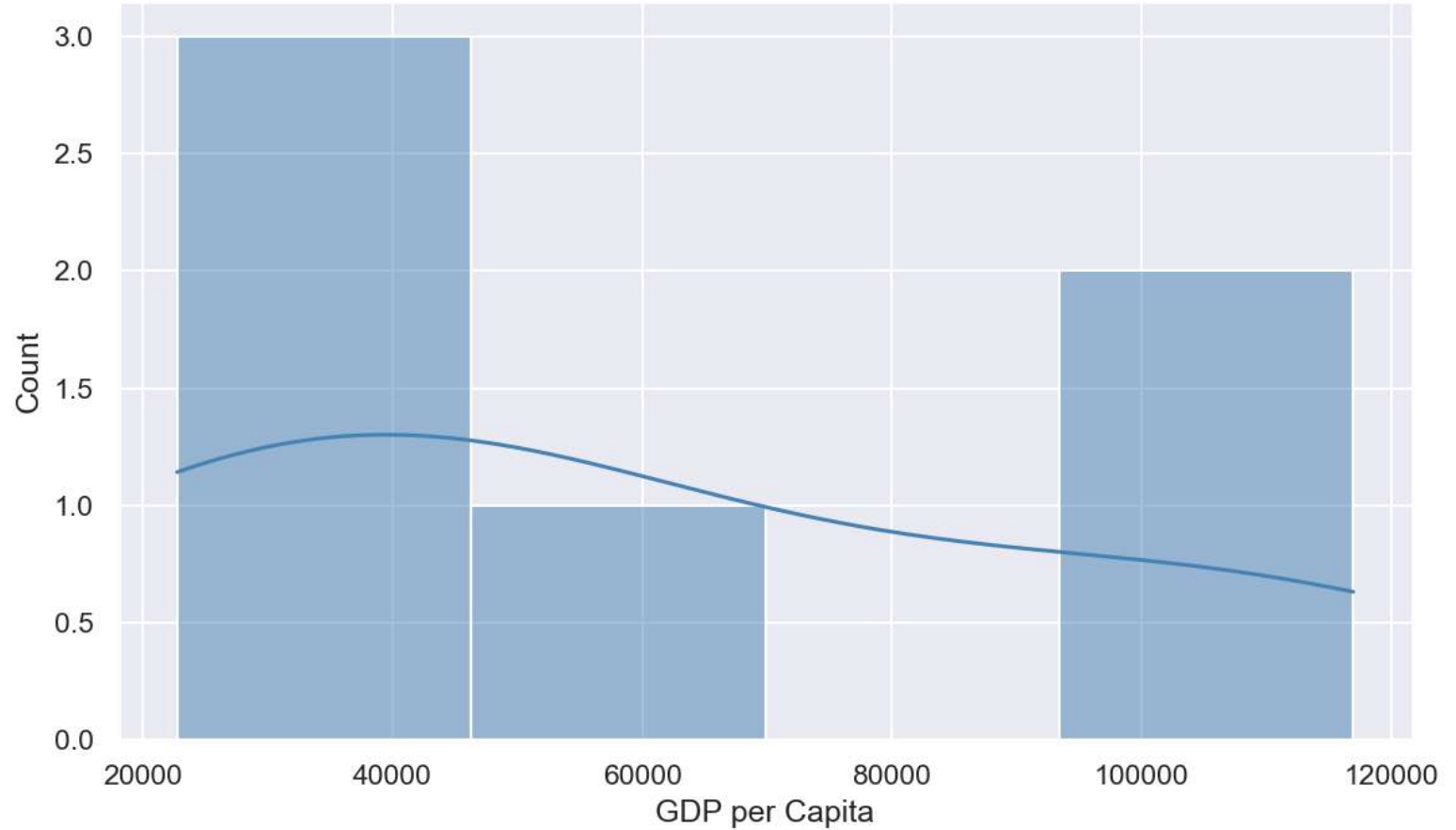
```
cm = confusion_matrix(y_test, y_pred) fig, ax = plt.subplots(figsize=(7, 5)) disp =
ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Rejected (N)', 'Approved
(Y)']) disp.plot(ax=ax, colorbar=True, cmap='Blues') ax.set_title(f'Confusion Matrix –
Accuracy: {acc*100:.2f}%', fontsize=13, fontweight='bold', pad=15) plt.tight_layout()
plt.savefig('oan_confusion_matrix.png', dpi=150, bbox_inches='tight') plt.show()
plt.close() print("■■ Confusion Matrix saved ■■")
```

```
coef_df = pd.DataFrame({ 'Feature': X.columns, 'Coefficient': model.coef_[0]
}).sort_values('Coefficient', ascending=False) fig, ax = plt.subplots(figsize=(9, 5))
colors = ['#2ecc71' if c > 0 else '#e74c3c' for c in coef_df['Coefficient']]
ax.barh(coef_df['Feature'], coef_df['Coefficient'], color=colors) ax.axvline(0,
color='black', linewidth=0.8) ax.set_title('Logistic Regression – Feature Coefficients',
fontsize=13, fontweight='bold') ax.set_xlabel('Coefficient Value') plt.tight_layout()
plt.savefig('loan_feature_importance.png', dpi=150, bbox_inches='tight') plt.show()
plt.close() print("■■ Feature importance plot saved ■■")
```

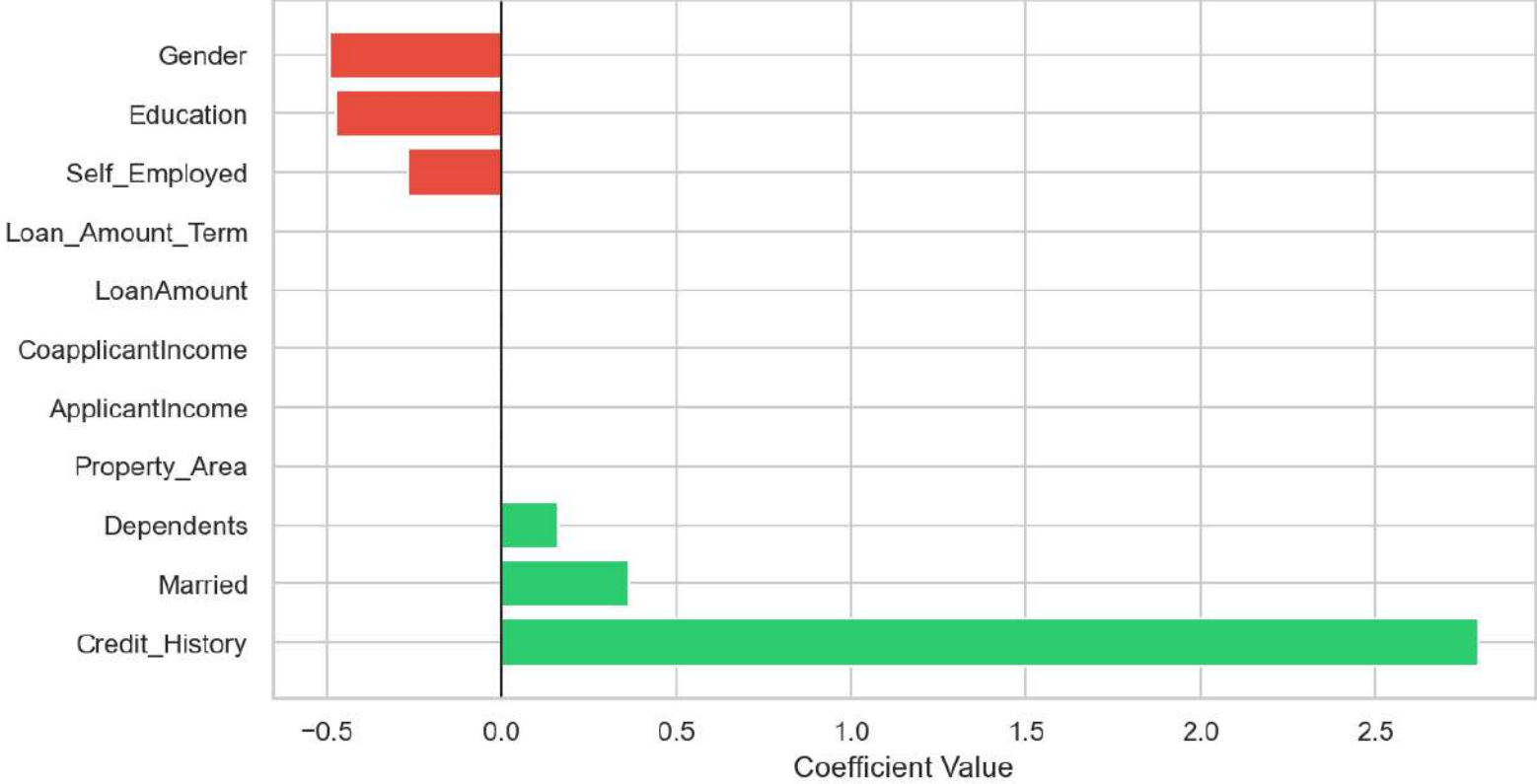
Total Cases by Continent



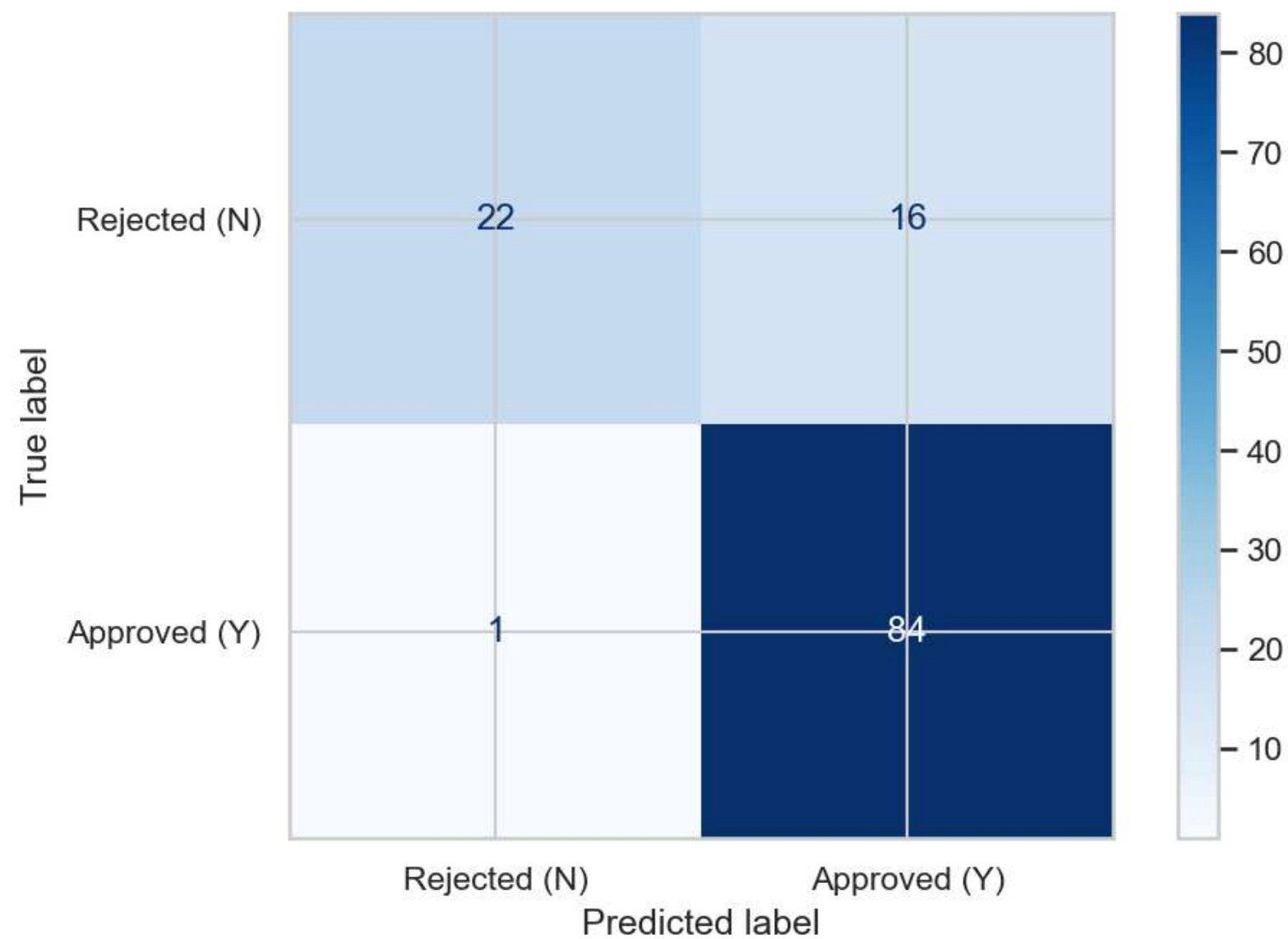
Distribution of GDP per Capita



Logistic Regression — Feature Coefficients



Confusion Matrix — Accuracy: 86.18%



Pairplot of df_groupby

